Busca Local e Algoritmos Genéticos no Sequenciamento de Manutenção de Ativos

Integrante 1* Integrante 2* Integrante 3*

Resumo

Comparamos três metaheurísticas – *Greedy Descent* com reinício aleatório (GD), *Simulated Annealing* (SA) e Algoritmo Genético (GA) – no problema de sequenciar n manutenções para minimizar o atraso ponderado total, em instâncias com $n \in \{12, 50, 150\}$. Todos os métodos recebem o mesmo orçamento de avaliações da função objetivo e as mesmas sementes aleatórias.

[Pendente] Integrante 3: uma frase com o resultado principal da comparação.

1 Formulação do Problema e Validação (T1)

[Pendente] Integrante 1: formulação de $h(\pi)$, tensão entre proximidade (t_{ij}) e urgência (d_j, w_j) e tabela de validação T1 ($h = 4366$ na identidade da instância pequena; $h = 39$ e $h = 22$ no exemplo da Figura 1 do enunciado).

Uma solução é uma permutação $\pi = (\pi_1, \ldots, \pi_n)$ dos ativos, com a equipe partindo da base $\pi_0 = 0$. Os instantes de conclusão e o custo são

$$C_k = C_{k-1} + t_{\pi_{k-1}\pi_k} + p_{\pi_k},$$
$$h(\pi) = \sum_{k=1}^{n} w_{\pi_k} \max\{0,\ C_k - d_{\pi_k}\}, \tag{1}$$

com $C_0 = 0$ e $t_{ij} = \text{round}\big(\|(x_i, y_i) - (x_j, y_j)\|_2\big)$.

2 Algoritmos e Estratégias de Busca

2.1 Vizinhanças e Avaliação Incremental (T2)

[Pendente] Integrante 1: cardinalidade de Swap ($n(n-1)/2$), Insertion ($(n-1)^2$) e 2-opt ($n(n-1)/2$); tabela comparativa T2 e *speedup* da avaliação incremental.

2.2 Greedy Descent com Random Restart

[Pendente] Integrante 1: variantes Best, First e Random Improvement, mecanismo de reinício e análise dos ótimos locais.

2.3 Simulated Annealing e Calibração Térmica

O *Simulated Annealing* [1, 2] é uma busca local estocástica que escapa de ótimos locais ao aceitar, com probabilidade controlada, movimentos que pioram a solução. A cada passo sorteia-se uniformemente um par ordenado (origem, destino) de posições distintas, o que define um vizinho π' da solução corrente π na vizinhança Insertion, e calcula-se $\Delta h = h(\pi') - h(\pi)$. A transição segue o critério de Metropolis, derivado da distribuição de Gibbs $P(\pi) \propto e^{-h(\pi)/T}$:

$$P(\pi \to \pi') = \begin{cases} 1, & \Delta h \le 0, \\ e^{-\Delta h/T}, & \Delta h > 0. \end{cases} \tag{2}$$

Como a proposta é simétrica (o movimento inverso tem a mesma probabilidade de ser sorteado), para T fixo a cadeia de Markov resultante tem a distribuição de Gibbs como distribuição estacionária. Quando $T \to \infty$, todo vizinho é aceito (*random walk*). Quando $T \to 0$, a massa de probabilidade se concentra nos mínimos de h, e a regra (2) se reduz a uma descida gulosa estocástica. O resfriamento leva a busca de um regime ao outro. A melhor solução global $\pi^\star$ é mantida à parte, porque o estado corrente pode terminar pior do que algum estado já visitado.

Calibração empírica de T_0. A escala de Δh depende fortemente de n. Os atrasos se acumulam ao longo da rota, de modo que reinserir um ativo desloca os instantes C_k de todo o sufixo. Uma temperatura fixa que funcione em $n = 12$ seria fria demais em $n = 150$, e uma adequada a $n = 150$ seria um *random walk* em $n = 12$. Por isso T_0 é calibrado em cada execução: a partir do estado inicial aleatório, sorteiam-se 200 vizinhos, calcula-se a média $\overline{\Delta h}^{+}$ apenas dos aumentos de custo ($\Delta h > 0$) e impõe-se que, no início, uma piora típica seja aceita com probabilidade $p_0 = 0{,}80$:

$$e^{-\overline{\Delta h}^{+}/T_0} = p_0 \implies T_0 = \frac{-\overline{\Delta h}^{+}}{\ln p_0} \approx 4{,}48\,\overline{\Delta h}^{+}. \tag{3}$$

A Tabela 1 mostra que $\overline{\Delta h}^{+}$ cresce cerca de 15× entre $n = 12$ e $n = 150$, e T_0 acompanha esse crescimento sem nenhum ajuste manual. As 200 avaliações de calibração são debitadas do orçamento B.

Esquema geométrico. A temperatura decai a cada avaliação, $T_{k+1} = \alpha\, T_k$, com

$$T_f = \rho\, T_0, \qquad \alpha = \left(\frac{T_f}{T_0}\right)^{1/L} = \rho^{1/L}, \tag{4}$$

em que $L = B - 201$ é o número de avaliações que restam após a avaliação do estado inicial e a calibração. Assim, T atinge T_f exatamente quando o orçamento se esgota. Adotamos $\rho = 10^{-3}$. Em testes-piloto com 3 a 4 sementes

*Departamento de Ciência da Computação – UFMG.

Tabela 1. Parâmetros do SA calibrados por instância (mediana entre as runs) e qualidade final. $\overline{\Delta h}^{+}$: piora média nos 200 vizinhos iniciais; $h^{\star}_{\min}$ e $\tilde{h}^{\star}$: mínimo e mediana do melhor h final.

n	B	runs	$\overline{\Delta h}^{+}$	T_0	T_f	α	$h^{\star}_{\min}$	$\tilde{h}^{\star}$
12	50 000	30	603	2 703	2,7	0,9998613	1 875	1 875
50	150 000	30	3 492	15 651	15,7	0,9999539	16 179	16 660
150	300 000	10	8 872	39 759	39,8	0,9999770	66 238	69 600

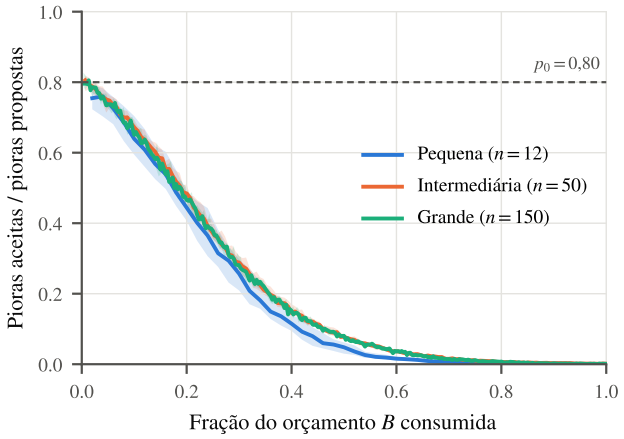


Figura 1. Fração das propostas de piora aceitas pelo SA em janelas de 1 000 avaliações, em função da fração consumida do orçamento. Linha: mediana entre as runs; faixa: intervalo interquartil.

(fora do conjunto oficial), fixar T_f pela mesma régua de T_0 (isto é, $T_f = -\overline{\Delta h}^{+}/\ln p_f$ com $p_f \in [10^{-2}, 10^{-4}]$) deixava o fim da busca quente demais em $n = 150$ (medianas acima de $1{,}5 \times 10^5$). Isso ocorre porque $\overline{\Delta h}^{+}$ é medido num estado aleatório, cujas pioras são muito maiores que as observadas perto de bons ótimos locais. Com $\rho = 10^{-3}$, T_f fica entre 2,7 e 39,8 unidades de custo (Tabela 1), da ordem das menores pioras possíveis. Valores menores ($\rho = 10^{-4}$) não trouxeram ganho. Também nos pilotos, Insertion superou Swap e 2-opt nas instâncias média e grande e foi adotada como vizinhança do SA.

Diagnóstico térmico. Em cada janela de 1 000 avaliações registramos a razão entre as pioras aceitas e as pioras propostas (Figura 1). As três curvas partem de $\approx 0{,}80$, o que confirma a calibração (3), e caem de forma suave e praticamente monotônica: a taxa mediana passa de 0,5 em $\approx 18\%$ do orçamento, de 0,2 em $\approx 35\%$ e de 0,05 entre 50% e 57%, e fica abaixo de 1% a partir de $\approx$ 66–75%. O primeiro terço da busca é, portanto, exploratório: a solução corrente perambula por regiões de custo alto, o que aparece como a descida lenta no início das curvas de convergência das instâncias média e grande (Figura 2). O terço final é, na prática, uma descida gulosa estocástica que refina a bacia escolhida. O fato mais relevante é que as três curvas quase se sobrepõem quando o eixo horizontal é normalizado por B. A calibração por $\overline{\Delta h}^{+}$ remove a dependência de escala, e a forma do resfriamento passa a depender apenas da fração do orçamento consumida, não de n. Não se observa nenhum dos dois modos de falha clássicos: nem um platô longo perto de 1 (temperatura alta demais, que desperdiça orçamento em passeio aleatório) nem uma queda abrupta para 0 no início (têmpera rápida demais, que congela a busca num ótimo local ruim).

2.4 Algoritmo Genético e Operadores de Permutação

[Pendente] Integrante 3: representação, seleção (torneio $k = 3$), cruzamento OX/PMX com exemplo numérico passo a passo, mutação, elitismo e parâmetros (N, P_c, P_m).

3 Metodologia Experimental e Protocolo T3

Orçamento por avaliações, não por tempo. O critério de parada de todos os algoritmos é um orçamento B de *avaliações da função objetivo*. Cada chamada a $h(\cdot)$ consome exatamente uma unidade, seja ela uma avaliação completa da Eq. (1) ou incremental. São contadas a avaliação da solução inicial, as 200 avaliações de calibração do SA, cada vizinho examinado pelo GD (inclusive os rejeitados) e cada indivíduo avaliado pelo GA. A contagem é centralizada em um único objeto `Avaliador` (módulo `src/avaliador.py`), pelo qual passam todas as chamadas a h e que levanta uma exceção ao se tentar ultrapassar B. Assim, nenhum algoritmo consegue gastar mais do que o orçamento.

[Pendente] Integrantes 1 e 3: confirmar que GD e GA avaliam h exclusivamente pelo `Avaliador` (nenhuma chamada direta a `Solution`).

O orçamento por avaliações é a métrica de esforço adotada na comparação porque independe da máquina, da carga do sistema e da qualidade da implementação. Dois métodos com o mesmo B examinaram o mesmo número de soluções candidatas. O tempo de parede (`time.perf_counter`) é registrado à parte, como medida secundária. Ele capta diferenças que o orçamento esconde: nem toda avaliação custa o mesmo (a incremental percorre só as posições $i, \ldots, n$), e as operações genéticas do GA têm custo próprio, que não passa por h. No SA, o tempo de parede por iteração (sorteio do vizinho, avaliação e aceitação) foi de 3,2 μs ($n = 12$), 5,5 μs ($n = 50$) e 14,8 μs ($n = 150$). O crescimento é sublinear em n porque parte do custo é fixo por iteração. O ganho da avaliação incremental, por outro lado, é modesto na nossa implementação: em $n = 150$, uma avaliação incremental custa 13,4 μs, contra 15,3 μs da completa. O laço em Python percorre só o sufixo, mas as cópias dos vetores C e H ainda custam $O(n)$.

Configurações. A Tabela 2 resume as nove combinações (3 algoritmos $\times$ 3 instâncias). Cada execução parte de uma permutação aleatória e grava, a cada 500 avaliações, o melhor h encontrado até ali, o que dá $B/500$ pontos por curva de convergência. O SA grava ainda T e a taxa de aceitação de pioras a cada 1 000 avaliações.

Controle por sementes. A run r de qualquer algoritmo usa a semente $s_r = 20\,260\,000 + r$, e toda a aleatoriedade

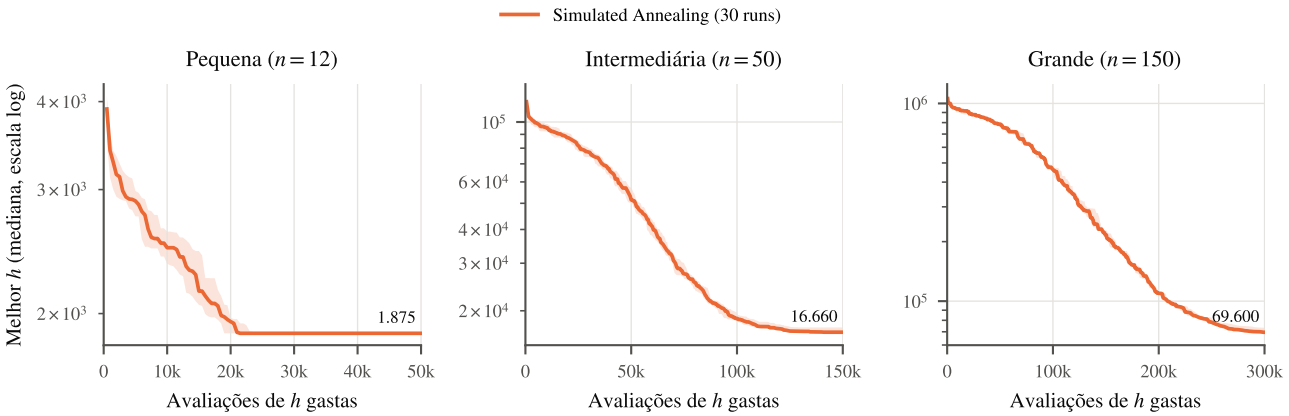


Figura 2. Curvas de convergência: mediana do melhor h (escala log) em função das avaliações gastas, com faixa interquartil, para as três instâncias. Tracejado: mediana do h médio da população do GA.

Tabela 2. Protocolo T3: orçamento e número de execuções independentes por instância.

Instância	n	B (avaliações)	Runs
Pequena	12	50 000	30
Intermediária	50	150 000	30
Grande	150	300 000	10

de uma execução (permutação inicial, sorteio de vizinhos, aceitação, operadores genéticos) vem de um único gerador `random.Random`(s_r) local. Nenhum algoritmo usa o gerador global do módulo `random`. Com isso, (i) qualquer execução é reproduzida bit a bit a partir de sua semente, e (ii) os três algoritmos usam as mesmas 30 (ou 10) sementes em cada instância. Usar a mesma semente, porém, não torna as runs pareadas: cada algoritmo consome o gerador de forma diferente, de modo que GD, SA e GA na run r não compartilham solução inicial nem trajetória. As amostras são, portanto, tratadas como independentes, e as comparações usam o teste de Mann–Whitney (Seção 4).

Execução e registro. As runs são executadas sequencialmente pelo orquestrador `scripts/experimentos_t3.py`, em um único processo, para que o tempo de parede não sofra contenção de CPU. Ele grava cada run em `resultados_t3.csv` imediatamente após seu término: algoritmo, instância, *run*, semente, $h^\star$ final, tempo de parede e histórico de convergência. Uma interrupção não perde as runs concluídas, e uma nova chamada retoma das pendentes. Os parâmetros calibrados do SA e o diagnóstico de aceitação vão para `parametros_sa.csv` e `diagnostico_sa.csv`. Todas as figuras e tabelas deste relatório são regeneradas a partir desses arquivos por `scripts/plot_graficos.py`. Os experimentos foram executados em Python 3.11.9, em um único núcleo de um Intel Core i7-1255U.

`figuras/boxplots.pdf`
ainda não gerada

Figura 3. Distribuição do melhor h final por algoritmo e instância.

4 Resultados Comparativos e Análise Estatística

[Pendente] Integrante 3: boxplots (3 painéis), p-valores de Mann–Whitney par a par (amostras independentes, ver Seção 3), discussão exploração × intensificação e experimento extra do GA (cruzamento ligado × desligado).

5 Discussão: Escala, Esforço Computacional e Conclusões

Efeito da escala no SA. Na instância pequena, o SA atinge o ótimo conhecido $h = 1\,875$ em todas as 30 runs, em mediana após ≈ 21 mil avaliações (43% de B). O restante do orçamento é gasto sem melhoria. Nas instâncias média e grande, ao contrário, a curva mediana só entra a 5% do valor final após 74% e 88% do orçamento, respectivamente, e a dispersão entre runs cresce: o desvio-padrão relativo vai de 0 para 4% ($n = 50$) e 4,5% ($n = 150$). Isso é coerente com o crescimento do espaço de busca ($n!$) e da vizinhança Insertion ($(n-1)^2 = 22\,201$ vizinhos em $n = 150$). Com $B = 300\,000$, o SA examina o equivalente a apenas ≈ 13 varreduras completas da vizinhança, e a curva mediana ainda desce quando o orçamento acaba. Isso sugere que, em $n = 150$, o fator limitante é o orçamento, e não o esquema de temperatura.

[Pendente] Integrantes 1 e 3: comparação de escala entre GD, SA e GA, custo por avaliação × tempo de parede dos três métodos, quando o GA compensa e conclusões gerais.

Tabela 3. Resumo do melhor h final por combinação algoritmo–instância, gap relativo $(\bar{h} - 1875)/1875$ na instância pequena, orçamento B e tempo médio de parede $\bar{t}$ por run.

Inst.	Alg.	runs	média	d.p.	mín.	mediana	gap	B	$\bar{t}$ (s)
Pequena	SA	30	1 875	0	1 875	1 875	0,00%	50 000	0,16
Intermediária	SA	30	16 944	673	16 179	16 660	–	150 000	0,83
Grande	SA	10	70 628	3 153	66 238	69 600	–	300 000	4,45

Referências

[1] S. Kirkpatrick, C. D. Gelatt e M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.

[2] N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller e E. Teller. Equation of state calculations by fast computing machines. *The Journal of Chemical Physics*, 21(6):1087–1092, 1953.

[3] S. Russell e P. Norvig. *Artificial Intelligence: A Modern Approach*. 4. ed., Pearson, 2021.